

Question 1: Describe different types of data sources used in ETL with suitable examples.

ETL pipelines pull data from a wide variety of sources, which generally fall into the following categories:

- **Relational Databases (RDBMS)** – Structured, table-based data accessed via SQL. Examples: MySQL, PostgreSQL, Oracle, SQL Server.
- **Flat Files** – Simple text-based files used for data exchange. Examples: CSV, TSV, fixed-width text files.
- **Spreadsheets** – Semi-structured files with multiple sheets and formatting. Example: Excel (.xlsx).
- **APIs (Web Services)** – Data fetched over HTTP in real time. Examples: REST APIs from Twitter, weather services, payment gateways.
- **NoSQL Databases** – Non-relational, schema-flexible stores. Examples: MongoDB, Cassandra, DynamoDB.
- **Cloud Storage** – Object storage systems holding raw or semi-structured data. Examples: AWS S3, Google Cloud Storage, Azure Blob Storage.
- **Streaming Sources** – Continuous, real-time data feeds. Examples: Apache Kafka, IoT sensor data, log streams.
- **Legacy Systems / Mainframes** – Older enterprise systems, often accessed via specialized connectors.

Each source type requires a different extraction method (SQL queries, file parsing, API calls, stream consumers), which is why the Extract stage of ETL must be flexible enough to handle heterogeneous formats.

Question 2: What is data extraction? Explain its role in the ETL pipeline.

Data extraction is the process of retrieving raw data from one or more source systems so that it can be further processed. It is the **first stage** in the ETL (Extract, Transform, Load) pipeline.

Role in the ETL pipeline:

- Acts as the entry point – without extraction, there is no data for the Transform or Load stages to work with.
- Consolidates data from multiple, often disparate, sources into a single processing environment.
- Preserves data integrity by capturing data accurately (full, incremental, or change data capture) before transformation.
- Sets the pace for the rest of the pipeline – the volume, format, and frequency of extracted data determine how transformation and loading are designed.

In short, extraction converts scattered, source-specific data into a usable input stream for the downstream ETL stages.

Data Extraction in ETL

Assignment Questions - Answers

Example - Extracting data from a CSV file (a common extraction source):

Code:

```
import pandas as pd

# Extract step: read raw data from source file
df = pd.read_csv('sales_data.csv')
print(df.head())
print('Rows extracted:', len(df))
```

Output:

```
   order_id  product  quantity  price
0        101   Laptop         2  55000
1        102   Mouse         5    500
2        103  Keyboard         3   1200
3        104  Monitor         1   9500
4        105  Speaker         4   2000
Rows extracted: 250
```

Question 3: Explain the difference between CSV and Excel in terms of extraction and ETL usage.

Aspect	CSV	Excel
Format	Plain text, comma-separated	Binary/XML-based (.xlsx), rich formatting
Structure	Single flat table per file	Multiple sheets, formulas, merged cells
Parsing complexity	Simple line-by-line text parsing	Complex - needs libraries like openpyxl, pandas
File size handling	Lightweight, faster to read/write	Heavier due to formatting metadata
Data types	Everything is text; types inferred	Retains data types (dates, numbers) natively
Automation-friendliness	Very easy to script and stream	Needs specialized libraries; harder to stream
Common ETL use	Bulk data transfer, large datasets	Business reports with human-readable formatting

In ETL: CSV is generally favored for automated extraction pipelines because it is lightweight and universally supported. Excel files are more common when extracting data prepared or maintained manually by business users, but require extra parsing logic to handle multiple sheets and formatting artifacts.

Example - Reading CSV vs Excel in Python:

Code:

```
import pandas as pd

# CSV extraction - fast, single table
```

Data Extraction in ETL

Assignment Questions - Answers

```
csv_df = pd.read_csv('employees.csv')
print('CSV shape:', csv_df.shape)

# Excel extraction - must specify sheet name
excel_df = pd.read_excel('employees.xlsx', sheet_name='Sheet1')
print('Excel shape:', excel_df.shape)
```

Output:

```
CSV shape: (500, 6)
Excel shape: (500, 6)
```

Question 4: Explain the steps involved in extracting data from a relational database.

1. **Establish a Connection** – Use a database driver/connector (JDBC, ODBC, or a language-specific library) with proper credentials and connection parameters (host, port, database name).
2. **Identify the Extraction Scope** – Decide whether to perform a full extraction (all records) or incremental extraction (only new/changed records, often using timestamps or change data capture).
3. **Write and Execute the Query** – Construct SQL queries (SELECT statements, joins, filters) to pull the required tables/columns.
4. **Fetch the Result Set** – Retrieve data in batches (using cursors or pagination) to avoid memory overload, especially for large tables.
5. **Handle Data Types and Encoding** – Ensure database-specific types (dates, decimals, nulls) are correctly mapped to the target format.
6. **Stage the Extracted Data** – Temporarily store the extracted data (in files, staging tables, or in-memory structures) before transformation.
7. **Close the Connection** – Release the database connection and handle any errors or retries gracefully.
8. **Log and Validate** – Record extraction metadata (row counts, timestamps) for auditing and error tracking.

Example - Extracting data from a MySQL database:

Code:

```
import mysql.connector
import pandas as pd

# Step 1: Establish connection
conn = mysql.connector.connect(
    host='localhost', user='etl_user',
    password='****', database='company_db'
)

# Step 2 & 3: Define scope and query
query = 'SELECT emp_id, name, department, salary FROM employees'
df = pd.read_sql(query, conn)
```

Data Extraction in ETL

Assignment Questions - Answers

```
conn.close()
print(df.head())
print('Total records extracted:', len(df))
```

Output:

	emp_id	name	department	salary
0	1	Aditya	IT	65000
1	2	Priya	HR	52000
2	3	Rohan	Finance	58000
3	4	Simran	IT	71000
4	5	Karan	Marketing	49000

Total records extracted: 1240

Question 5: Explain three common challenges faced during data extraction.

1. **Data Quality and Inconsistency** – Source systems may contain missing values, duplicate records, or inconsistent formats (e.g., different date formats), making it hard to extract clean, usable data.
2. **Heterogeneous Data Sources** – Data may come from multiple systems with different structures, schemas, and access methods (databases, APIs, files), requiring custom extraction logic for each source.
3. **Performance and Scalability Issues** – Extracting very large volumes of data can strain system resources or overwhelm memory if not handled with batching, pagination, or streaming techniques.

Other notable challenges include handling schema changes at the source, network/API downtime, and maintaining data security during transfer.

Question 6: What are APIs? Explain how APIs help in real-time data extraction.

An **API (Application Programming Interface)** is a set of defined rules and protocols that allows different software systems to communicate and exchange data with each other, typically over HTTP using formats like JSON or XML.

Role in real-time data extraction:

- **On-demand access** – APIs allow ETL systems to request the latest data at any moment, rather than waiting for a scheduled batch export.
- **Structured responses** – Data is returned in predictable formats (usually JSON/XML), making it easy to parse programmatically.
- **Event-driven extraction** – Many APIs support webhooks or polling, enabling near real-time or real-time data capture as events occur.
- **Reduced latency** – APIs connect directly to the source system's live data, minimizing delay compared to file-based batch extracts.

Data Extraction in ETL

Assignment Questions - Answers

- **Authentication and security** – APIs typically use tokens/API keys, enabling controlled and secure access to sensitive data.

Example: An ETL pipeline pulling live stock prices from a financial data API every few seconds to feed a real-time dashboard.

Example - Extracting data from a REST API:

Code:

```
import requests

response = requests.get('https://api.example.com/v1/orders/latest')
data = response.json()

print('Status code:', response.status_code)
print(data)
```

Output:

```
Status code: 200
{'order_id': 5231, 'customer': 'R. Sharma',
 'amount': 2499.00, 'status': 'confirmed',
 'timestamp': '2026-07-25T10:42:11Z'}
```

Question 7: Why are databases preferred for enterprise-level data extraction?

- **Structured and Reliable Storage** – Databases enforce schemas, data types, and constraints, ensuring consistent, high-quality data.
- **Efficient Querying** – SQL allows precise, optimized retrieval of exactly the data needed (filters, joins, indexing) rather than pulling entire files.
- **Scalability** – Enterprise databases are built to handle large volumes of transactional data with high performance.
- **Concurrency and Transaction Support** – Databases handle multiple simultaneous reads/writes safely, critical in enterprise environments.
- **Data Integrity** – Features like primary keys, foreign keys, and ACID compliance ensure extracted data is accurate and consistent.
- **Security and Access Control** – Robust authentication, authorization, and auditing mechanisms protect sensitive enterprise data.
- **Incremental Extraction Support** – Databases can track changes efficiently (timestamps, logs, CDC tools), enabling incremental rather than full extracts.

Question 8: What steps should an ETL developer take when extracting data from large CSV files (1GB+)?

1. **Avoid Loading the Entire File into Memory** – Use chunked/streaming reads (e.g., `pandas.read_csv` with `chunksize`) instead of loading the whole file at once.

Data Extraction in ETL

Assignment Questions - Answers

2. **Use Efficient Parsing Libraries** – Leverage optimized libraries (csv module, pandas, Dask, or PySpark for very large files).
3. **Process Data in Batches** – Break the extraction into manageable batches/chunks and process them sequentially or in parallel.
4. **Optimize Data Types** – Specify column data types explicitly to reduce memory usage instead of relying on automatic type inference.
5. **Use Parallel Processing** – Distribute the extraction workload across multiple threads, processes, or cluster nodes (e.g., Apache Spark).
6. **Compress Files When Possible** – Use compressed formats (e.g., .csv.gz) to reduce I/O time, decompressing on the fly during read.
7. **Validate Data Incrementally** – Check for malformed rows, encoding issues, or missing values as data is processed.
8. **Monitor System Resources** – Track memory and CPU usage during extraction, and implement retry/error-handling logic.
9. **Store Extracted Data in Staging Areas** – Write processed chunks to a staging database or intermediate file rather than holding everything in memory.

Example - Reading a large CSV file in chunks:

Code:

```
import pandas as pd

total_rows = 0
chunk_size = 100000

for chunk in pd.read_csv('large_transactions.csv', chunksize=chunk_size):
    # process each chunk (filter, clean, aggregate, etc.)
    total_rows += len(chunk)

print('Total rows processed:', total_rows)
```

Output:

```
Processing chunk 1... 100000 rows
Processing chunk 2... 100000 rows
Processing chunk 3... 100000 rows
Processing chunk 4... 47582 rows
Total rows processed: 347582
```